

FoodAI

Application web de reconnaissance de plats avec:

- authentification Google (Supabase Auth)
- profil utilisateur (préférences + consentement image)
- historique des scans
- favoris
- affichage recette avec ajustement des portions

1. Structure du projet

- ``/Users/yunes/Documents/New project/AM1_projet/app/web/`` : backend FastAPI + frontend (templates/static)
- ``/Users/yunes/Documents/New project/AM1_projet/app/ml/`` : prédiction image (PyTorch)
- ``/Users/yunes/Documents/New project/AM1_projet/app/core/`` : logique recettes/unités/scaling
- ``/Users/yunes/Documents/New project/AM1_projet/app/data/recipes.json`` : base recettes
- ``/Users/yunes/Documents/New project/AM1_projet/data/food/`` : dataset d'entraînement

2. Pré-requis recommandés

- Python 3.10 (recommandé)
- pip récent
- Supabase projet configuré

Pourquoi Python 3.10:

- évite plusieurs conflits rencontrés en 3.9 (packages ML)
- installation ``torch/torchvision`` plus stable

3. Installation propre (recommandée)

Depuis ``/Users/yunes/Documents/New project/AM1_projet``:

```
```bash
python3.10 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

### ## 4. Variables d'environnement

Créer ``/Users/yunes/Documents/New project/AM1_projet/.env``:

```
```env
SUPABASE_URL=...
SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_ROLE_KEY=...
ADMIN_TOKEN=...
SUPABASE_IMAGE_BUCKET=scan-images
```
```

Notes:

- ``SUPABASE_SERVICE_ROLE_KEY`` sert aux endpoints admin.
- ``ADMIN_TOKEN`` protège les routes admin custom.

### ## 5. Lancer l'application

```
```bash
source .venv/bin/activate
python -m app.main --mode web
```

- Interface: `[http://localhost:8000](http://localhost:8000)`

- Docs API: http://localhost:8000/docs

6. Fonctionnalités actuellement validées

- Login Google avec persistance session
- Overlay de connexion + UI utilisateur (menu haut droite)
- Préférences utilisateur:
 - prénom, nom
 - restrictions alimentaires
 - consentement stockage image
- Historique:
 - lecture
 - suppression
 - restauration d'un résultat
- Favoris:
 - ajout
 - lecture
 - suppression
 - restauration d'un résultat avec recette sauvegardée
- Scan image -> prédiction top-k -> recette
- Ajustement dynamique des portions côté interface (rescaling affiché en direct)

7. Correctif portions (important)

Le comportement attendu est:

- après un scan réussi, changer `Servings` met à jour immédiatement les quantités affichées,
- sans rescanner l'image.

Implémentation actuelle:

- conservation d'une recette base en mémoire frontend,
- application d'un ratio `requested\_servings / base\_servings`,
- recalcul des quantités formatées dans l'affichage.

Fichiers concernés:

- `/Users/yunes/Documents/New project/AM1\_projet/app/web/static/js/app.js`
- `/Users/yunes/Documents/New project/AM1\_projet/app/web/templates/index.html`

8. Troubleshooting

```
### Erreur `No module named 'dotenv'`  
```bash
```

```
source .venv/bin/activate
python -m pip install python-dotenv
```

```
Erreur `No module named 'torch'`
```

Tu n'es probablement pas dans le bon venv.

```
```bash  
source .venv/bin/activate  
python -c "import torch; print(torch.__version__)"
```

Si erreur:

```
```bash  
python -m pip install torch torchvision
```

```
L'app ne recharge pas les changements JS
```

- redémarrer le backend
- hard refresh navigateur (`Cmd + Shift + R`)

## ## 9. Tests rapides manuels

1. Connexion Google OK

2. Upload image plat connu (ex: pizza)
3. Vérifier top-k + recette
4. Changer `Servings` de 2 à 4 et vérifier le rescaling affiché
5. Ajouter en favori, rouvrir favori, vérifier affichage
6. Ouvrir historique, vérifier lecture/suppression

## 10. Notes Git

- Branche de travail actuelle: `feature/leo`
- Remote: `origin -> <https://github.com/Hisok4YEZ/FoodAI.git>`